Nubra TradeGuard — Product Case Study

Product Management Case Study | Trade Execution Clarity & Order Reliability

> Note on evidence: treats the core problem as a hypothesis to be validated, not a proven fact. Findings labelled Evidence / Inference / Hypothesis throughout.

---

01. Product Context

About Nubra

Nubra is an all-in-one trading platform focused on active traders, including options and strategy-focused users. Its product includes advanced option-chain data, multi-strike charts, multi-leg strategy builders, Smart Flexi Orders, smart watchlists, and other trading capabilities. The platform positions itself around depth and speed for derivatives traders rather than beginner, buy-and-hold investing.

Why this problem?

Nubra's product is strongly oriented toward active and derivatives traders, a segment where order execution is time-sensitive and often complex. Multi-leg strategies involve multiple execution states happening close together. A trader may need to understand, quickly and under pressure, which parts of their trade went through and which did not. This is not a claim that Nubra's execution system is unreliable. It is an observation that the complexity of the workflow raises the stakes on clarity, making execution clarity and recovery a potential product opportunity worth investigating.

Opportunity

TradeGuard explores a clarity and recovery layer around Nubra's existing trade execution experience. Rather than changing the underlying trading engine, the product helps traders:

1. Understand the current state of their trade.
2. Diagnose why an unexpected state occurred.
3. Act by taking an appropriate next step.

---

02. Research & Evidence

What we found

Type	Finding	Why it matters
Evidence	Nubra supports multi-leg F&O strategy orders and strategy-based trading workflows.	Multiple legs can create different execution states that traders may need to understand.
Evidence	Nubra's order infrastructure supports states such as open, executed, rejected, cancelled and expired.	Traders need clear visibility into what happened to an order.
Evidence	Public Nubra feedback contains both positive reports (fast order confirmation, smooth performance) and reports of execution or exit issues.	The experience should be investigated rather than assuming it is universally problematic.
Inference	Active F&O traders are likely to be particularly sensitive to execution clarity because trading decisions can be time-sensitive.	Execution clarity may be especially important during high-pressure trading situations.
Hypothesis	Traders may struggle to understand and recover from unexpected order states.	This is the core problem we will validate through the proposed product experience.

Sources

- Nubra Product Intern JD
- Nubra official product / API documentation
- Nubra App Store reviews
- Public industry reporting on trading-system disruptions

No fabricated URLs, quotes, or statistics are used.

---

03. Problem Definition

Problem Hypothesis (To Be Validated)

“Intermediate options traders on Nubra who place margin-dependent and multi-leg orders need a clearer, real-time way to review, track and recover from their trades because ambiguity around order state, rejection reasons, and per-leg execution status during high-pressure moments can leave them in unintended positions, erode confidence, and reduce trust in the platform.”

Status: Problem Hypothesis — To Be Validated

Why this matters

- Trading decisions can be time-sensitive.
- Unexpected order states can create uncertainty.
- Users need to understand what happened.
- Users need to understand their current position.
- Users need to know what action they can take.
- This becomes more important for complex F&O workflows with multiple legs.

Research Question

“How might Nubra help traders understand and recover from unexpected trade-execution states without adding unnecessary friction to normal trading?”

---

04. Target User

Primary User: Intermediate F&O Trader

Attribute	Description
Experience	Understands basic equity / options trading
Trading style	Actively trades F&O
Product usage	Uses charts, option chains and strategy / order tools

Goal	Execute and manage trades quickly and confidently
Frustration	Unclear order status or unexpected execution state
Need	Immediate understanding and actionable recovery

Job-to-be-Done

“When my trade doesn’t execute as expected, I want to immediately understand what happened and what action I can take, so that I can protect / manage my position confidently.”

---

05. Current User Journey

Choose Strategy → Configure Trade → Review Order → Place Order → Order Processing → Success OR Unexpected State → Understand Problem → Decide What To Do → Recover

Stage	User need	Potential friction
Configure	Verify intended trade	Complex information
Review	Confirm order	Important details may require attention
Execute	Know what is happening	Uncertainty during processing
Unexpected State	Understand why	Technical or unclear error
Recovery	Take action quickly	User may not know the next step
Exit	Confirm position is closed	High consequence if unclear

Key Insight

“The biggest opportunity is not adding another trading feature. It is improving the clarity and actionability of the existing execution experience when something unexpected happens.”

Understand → Diagnose → Act

---

06. TradeGuard: Proposed Solution

TradeGuard is a proposed **clarity and recovery layer** around Nubra's existing trade execution experience. It does not replace the trading engine and it does not make trading recommendations. Its purpose is to help traders **understand** the state of their trade, **diagnose** unexpected outcomes, and **act** by taking an appropriate next step.

1. Pre-Trade Review

Purpose: Allow the trader to quickly verify what they are about to execute before placing the trade.

Shows: Instrument, Buy / Sell, Quantity, Order type, Price, Margin requirement, Estimated charges, Number of legs (for multi-leg strategies).

Primary CTA: "Confirm & Place Order"

This screen stays **compact**. The goal is fast verification, not information overload. Secondary details can be expanded on demand.

2. Execution Status

Purpose: Give the trader a clear view of what is happening after placing an order.

Possible states: **Processing, Accepted, Partially Filled, Executed, Rejected, Cancelled.**

For multi-leg strategies, the state of **each leg** is shown separately.

Strategy: Bull Call Spread

Leg 1 — BUY 100 CE — Executed
Leg 2 — SELL 110 CE — Pending

The user should immediately understand the current state of the whole strategy at a glance.

3. Failure Explanation

Purpose: Translate an unexpected order state into a simple, human-readable explanation.

Shows: **What happened, Why it happened, Current impact, What the user can do next.**

Order Rejected

Reason:
Insufficient available margin.

Current status:
Order not executed.

Available actions:
- Add Funds
- Modify Order
- Cancel / Exit (where applicable)

TradeGuard explains the operational situation. It does **not** provide financial advice or recommend a trading decision.

4. Recovery Actions

Purpose: Help the user take an appropriate operational action after an unexpected state.

Possible actions: Retry / modify order (where appropriate), Add funds, Review affected leg, Exit an existing position (where applicable), Contact support for unresolved technical issues.

TradeGuard provides **operational clarity, not investment recommendations**. The final trading decision always stays with the user.

---

07. MVP & Feature Prioritization

The MVP should solve the **core problem**, helping traders understand and recover from unexpected execution states, without attempting to redesign Nubra's entire trading platform. Scope is kept deliberately tight so the project stays shippable and testable.

Priority	Feature	Reason
Must Have	Pre-trade summary	Lets the user verify the intended trade before committing real money.
Must Have	Clear order status	Removes the core uncertainty of "what happened to my order?"

Must Have	Clear rejection / failure explanation	Turns a confusing failure into an understandable situation.
Must Have	Multi-leg execution visibility	Prevents unintended partial positions in strategy trades.
Must Have	Actionable recovery options	Closes the loop from confusion to resolution.
Should Have	Margin visibility	Prevents many rejections before they happen; supports diagnosis.
Should Have	Estimated charges	Improves pre-trade confidence and transparency.
Should Have	Execution timestamps	Helps users reconstruct what happened and when.
Could Have	Educational explanations	Helps less-experienced users learn, but not essential to the core loop.
Could Have	Personalized execution insights	Adds value over time but depends on usage data maturity.
Won't Have (MVP)	Stock / option recommendations	Out of scope; introduces advisory and regulatory complexity.
Won't Have (MVP)	Price prediction	Not the problem being solved; high risk, low relevance.
Won't Have (MVP)	Automated trading decisions	Removes user control; increases risk.
Won't Have (MVP)	Portfolio optimization	Different problem space entirely.
Won't Have (MVP)	AI-generated financial advice	Advisory territory; explicitly outside TradeGuard's purpose.

Why the MVP is limited to these capabilities

The Must-Haves map directly to the three-part philosophy: **Understand** (pre-trade summary, order status), **Diagnose** (failure explanation, multi-leg visibility), and **Act** (recovery options). Everything in Should / Could / Won't either supports that loop or sits outside it.

Anything that recommends *what* to trade is deliberately excluded, because it changes TradeGuard from an operational-clarity layer into an advisory product, with all the risk and regulatory weight that carries.

Guiding principle:

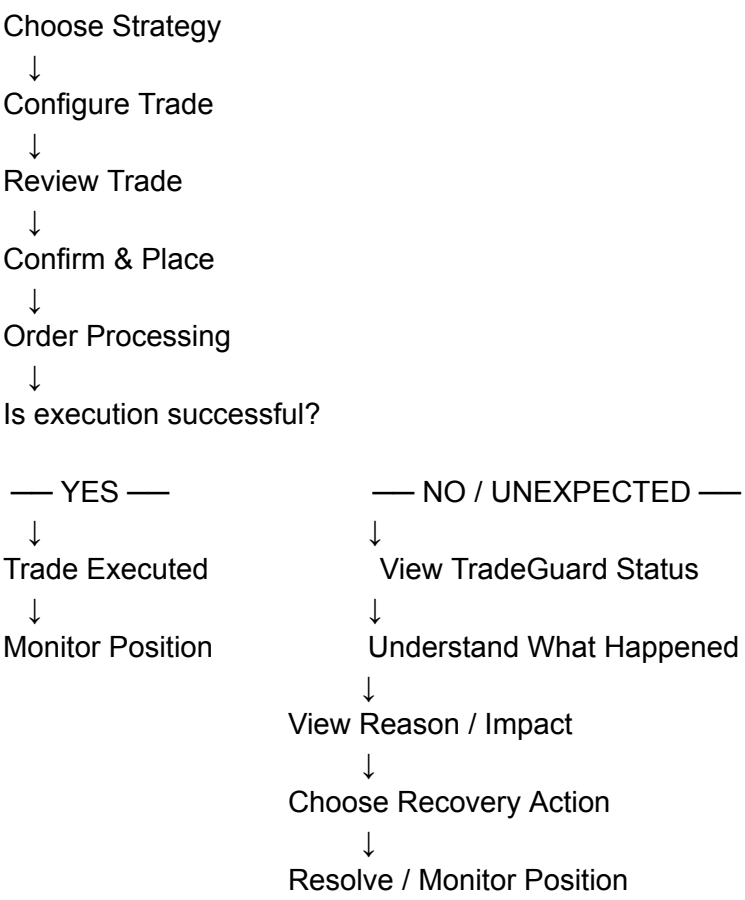
> "Improve execution clarity without increasing unnecessary friction during normal, successful trades."

The features that only appear when something is unexpected (failure explanation, recovery) must stay invisible on the happy path. A trader whose order fills cleanly should barely notice TradeGuard is there.

---

08. Primary User Flow

Core flow



Multi-leg flow

Build Multi-Leg Strategy
↓
Review All Legs
↓
Place Strategy
↓
Track Individual Legs
↓
All Executed OR Unexpected State
↓
Identify Affected Leg
↓
View Reason
↓
Take Appropriate Operational Action

The two flows share the same spine. The difference is that the multi-leg flow adds **per-leg tracking**, so a trader can see exactly which part of a strategy is stuck rather than treating the whole order as one opaque success or failure.

---

09. Product Principles & Trade-offs

Product Principles

1. **Clarity over complexity.**
2. **Explain the problem before showing the action.**
3. **Show critical information first**, expand detail on demand.
4. **Do not slow down normal, successful trades.**
5. **Never present financial advice as an operational recommendation.**

Key Trade-offs

More information vs simplicity

Showing too much information may increase cognitive load, especially in a time-pressured moment.

Decision: Show essential, decision-critical information first and allow secondary details to be expanded.

Safety / clarity vs speed

Additional confirmation steps can slow experienced traders who trade frequently.
Decision: Keep the review compact and focused on decision-critical information, so it reads in a glance rather than a study.

Technical error vs user-friendly explanation

Backend error messages and codes rarely help a trader understand what to do next.
Decision: Translate technical failure states into simple explanations paired with actionable operational options.

Automation vs user control

Automatically changing or closing a position could create additional, unintended risk.
Decision: TradeGuard **explains and surfaces options** but keeps the final trading action under user control.

---

10. Product Requirements Document (PRD)

10.1 Product Overview

TradeGuard is a proposed **clarity and recovery layer** around Nubra's existing trade execution experience. It helps active F&O traders **understand** the state of their trade, **diagnose** unexpected execution states, and **act** by taking appropriate operational next steps.

TradeGuard does **not**: replace Nubra's trading engine, make investment recommendations, predict prices, or automatically decide whether a trader should buy, sell, or exit.

10.2 Problem

"Intermediate options traders on Nubra who place margin-dependent and multi-leg orders need a clearer, real-time way to review, track and recover from their trades because ambiguity around order state, rejection reasons, and per-leg execution status during high-pressure moments can leave them in unintended positions, erode confidence, and reduce trust in the platform."

Status: Problem Hypothesis — To Be Validated. This PRD proposes a solution to test the hypothesis, not a response to a confirmed problem.

10.3 Target User

Intermediate F&O Trader (defined in Section 04). Understands basic equity / options trading, actively trades F&O, uses charts, option chains, and strategy / order tools.

Job-to-be-Done: “When my trade doesn’t execute as expected, I want to immediately understand what happened and what action I can take, so that I can protect / manage my position confidently.”

10.4 Product Goal

Primary goal: “Improve clarity and recovery during unexpected trade-execution states without materially slowing down normal successful trading.”

Supporting objectives:

1. Help users quickly understand the current state of their trade.
2. Explain unexpected order states in simple, actionable language.
3. Help users identify appropriate operational next steps.

10.5 Non-Goals

Non-goal	Reason
Stock / option recommendations	Advisory territory; outside operational-clarity scope.
Price prediction	Not the problem being solved; high risk, low relevance.
Automated trading decisions	Removes user control; increases risk.
Portfolio optimization	Different problem space entirely.
Financial advice	Regulatory weight; explicitly outside TradeGuard’s purpose.
Rebuilding Nubra’s order execution engine	TradeGuard is a layer on top, not a replacement.

10.6 User Stories

1. As an intermediate F&O trader, I want to review the key details of my trade before I place it, so that I can confirm I am executing the trade I intended.
2. As an intermediate F&O trader, I want to see the current status of my order after placing it, so that I know whether it went through.
3. As an intermediate F&O trader, I want to see the status of each leg in a multi-leg strategy, so that I know exactly which parts executed and which did not.
4. As an intermediate F&O trader, I want to understand why an order was rejected, so that I am not left guessing what went wrong.
5. As an intermediate F&O trader, I want to understand the current impact of an unexpected state, so that I know whether I am exposed or safe.
6. As an intermediate F&O trader, I want to see the operational actions available to me after a problem, so that I can respond quickly.
7. As an intermediate F&O trader, I want to see whether my recovery action worked, so that I know the situation is resolved.
8. As an intermediate F&O trader, I want to confirm the final state of my trade or position, so that I can step away with confidence.

10.7 Functional Requirements

ID	Requirement	Priority	Rationale
FR-01	Display a compact pre-trade summary before the user confirms an order.	Must Have	Lets the user verify the intended trade before committing real money.
FR-02	The summary should display instrument, direction, quantity, order type and price.	Must Have	These are the decision-critical fields for verifying a trade.
FR-03	For applicable trades, the summary should display relevant margin and estimated charge information.	Should Have	Surfaces the most common rejection cause (margin) before placement.
FR-04	Display the current order state after order placement.	Must Have	Directly removes the core “what happened to my order?” uncertainty.
FR-05	Support clear user-facing states such as Processing, Accepted, Partially Filled, Executed, Rejected and Cancelled where applicable.	Must Have	A shared, human-readable state vocabulary is the basis of clarity.
FR-06	For multi-leg strategies, display the state of each individual leg.	Must Have	Prevents unintended partial positions in strategy trades.

FR-07	When an order enters an unexpected state, provide a human-readable explanation.	Must Have	Turns a confusing failure into an understandable situation.
FR-08	The explanation should distinguish between what happened, why it happened, and the current impact.	Must Have	Structured explanation supports fast, confident diagnosis.
FR-09	Display appropriate operational recovery options based on the order state.	Must Have	Closes the loop from confusion to resolution.
FR-10	Recovery actions should require explicit user confirmation before any trade-changing action.	Must Have	Keeps the final trading decision under user control; prevents accidental risk.
FR-11	Update the displayed state when the underlying order state changes.	Must Have	A stale status is worse than no status in a time-sensitive moment.
FR-12	Provide a clear final state after the recovery workflow is completed.	Should Have	Lets the user confirm resolution and step away with confidence.

10.8 Acceptance Criteria

FR-01 — Pre-Trade Summary: Given the user has configured a trade, when they proceed to confirmation, then the system displays the relevant trade details before the order is placed.

FR-04 — Order Status: Given an order has been submitted, when its state changes, then the user can see the latest available state.

FR-06 — Multi-Leg Visibility: Given the user has placed a multi-leg strategy, when one or more legs have different states, then the system clearly identifies the state of each leg.

FR-07 — Failure Explanation: Given an order is rejected, when the user opens the order details, then the system displays a human-readable reason where available.

FR-09 — Recovery Actions: Given an unexpected state occurs, when recovery options are available, then the system displays relevant operational actions without making an investment recommendation.

FR-10 — User Confirmation: Given a recovery action could modify or exit a position, when the user selects that action, then the system requires explicit confirmation before proceeding.

10.9 Edge Cases

Scenario	Expected behavior
Insufficient margin	Show a rejection with a margin explanation and operational actions (add funds, modify order).
Order rejected	Show a Rejected state with a human-readable reason where available and relevant recovery options.
Order partially filled	Show a Partially Filled state indicating filled vs remaining quantity, with options to modify or cancel the remainder.
Multi-leg strategy with different leg states	Show each leg's state separately and clearly identify the affected leg(s).
Market closed	Prevent or clearly flag the action, explaining that the market is closed and when action may be possible.
Network interruption	Indicate that the connection was interrupted and that the latest status may take a moment; do not assert failure.
Order cancellation	Show a Cancelled state and confirm no position was opened as a result.
Session timeout	Prompt re-authentication and preserve / re-fetch the latest known order state rather than assuming an outcome.
Order state takes longer than expected	Show a delayed-status message and continue checking for an update rather than defaulting to failure.
User attempts duplicate order	Surface that a similar recent order exists and ask for explicit confirmation before proceeding.

10.10 Error / Empty / Loading States

- **Loading:** "Checking order status…"
- **Delayed status:** "Order status is taking longer than expected. We're checking for an update."
- **Unknown status:** "We couldn't confirm the latest status. Please refresh or check your orders."
- **Rejected:** "Order rejected" + reason where available + relevant operational action.
- **Network interruption:** "Connection interrupted. Your latest order status may take a moment to update."

Critical rule: Do not tell users that an order failed unless the system has **confirmed** the failure. An incorrect “failed” message could cause a trader to re-place an order that actually went through, doubling their position.

10.11 Scope

MVP: Pre-trade review, Order status, Multi-leg status visibility, Failure explanation, Recovery actions.

Future: Personalized execution insights, More advanced explanations, Additional monitoring capabilities.

10.12 Dependencies & Risks

Dependency / Risk	Why it matters	Mitigation
Reliable order-state data	TradeGuard’s clarity is only as good as the state data it displays.	Design around confirmed states; show “unknown” honestly rather than guessing.
Timely status updates	Stale status in a time-sensitive moment is actively harmful.	Refresh on state change (FR-11); show a delayed-status message when updates lag.
Accuracy of failure reasons	A wrong reason erodes the trust the product is meant to build.	Only display a reason where available; fall back to a neutral message otherwise.
Avoiding additional friction	Extra steps could annoy frequent traders and hurt adoption.	Keep the pre-trade review compact; keep TradeGuard invisible on the happy path.
Users misreading operational info as financial advice	Could create liability and user harm.	Consistent operational-only language; never recommend a trading decision.

10.13 Success Criteria

The MVP will be considered promising if testing shows that users can:

1. Understand their order state quickly.
2. Identify why an unexpected state occurred.
3. Identify the appropriate operational next step.
4. Complete the recovery workflow successfully.

5. Do this without materially increasing friction during normal, successful trades.

Detailed KPIs and experiment design are defined in Section 11.

---

11. Metrics & Experiment

11.1 North Star Metric

North Star Metric: Unexpected-State Resolution Rate

Definition: Percentage of unexpected trade-execution cases where the user reaches a resolved / understood state after encountering an unexpected order state.

Why this is the North Star: TradeGuard is designed to help users **understand, diagnose and act**. Feature usage alone is not success. A user opening a failure screen is not, by itself, a positive outcome, it usually means something went wrong. What matters is whether that user then understands the situation and reaches an appropriate resolved state. The North Star measures the *outcome of the loop*, not the entry into it.

Why we deliberately avoid these as the North Star:

- **TradeGuard adoption** measures exposure, not whether the user was actually helped.
- **Number of clicks** can go up when users are confused, so more is not better.
- **Number of screens viewed** rewards a heavier experience, which is the opposite of what we want on a time-sensitive path.

11.2 Supporting Metrics

Metric	Definition	Why it matters
Order-State Comprehension Rate	Percentage of tested users who correctly identify the current state of their order.	Direct measure of whether the “Understand” step works.
Recovery Completion Rate	Percentage of eligible unexpected states where the user successfully completes the selected recovery flow.	Measures whether the “Act” step actually resolves the situation.
Time to Understand	Time from unexpected-state display until the user correctly identifies what happened.	Shorter time means less panic and clearer communication.

Time to Resolution	Time from unexpected-state display until the trade reaches a confirmed resolved state.	Captures the full exception journey, end to end.
Normal Trade Completion Time	Median time to complete a normal, successful order.	Confirms TradeGuard is not taxing the happy path.
Unexpected-State Abandonment Rate	Percentage of unexpected states the user leaves without reaching resolution.	High abandonment signals confusion or a dead end.
Support Contact Rate	Percentage of relevant execution-state cases that still require support contact.	A fallback measure; good clarity should reduce this over time.
User Confidence	Post-task rating of how confident the user is that they understood the final state of their trade.	Qualitative check on trust, the ultimate goal of the product.

Business metrics such as **retention or revenue** would be longer-term measures. They should not be treated as direct causal outcomes of this MVP without further evidence.

11.3 Guardrail Metrics

TradeGuard should solve the **exception path** without damaging the **happy path**.

Guardrail	What we monitor	Why
Normal Trade Completion Time	Median time to place a normal, successful order.	The most important guardrail; clarity must not slow routine trading.
Normal Trade Abandonment Rate	Drop-off during normal, successful order placement.	Detects friction introduced into the common case.
Duplicate Order Attempts	Repeated placement of the same order.	Signals unclear status that makes users re-try.
Incorrect Recovery Actions	Recovery flows where users choose an inappropriate or unsuccessful action.	Detects a confusing or misleading recovery experience.
Incorrect or Stale Order Status	Cases where the displayed status is wrong or out of date.	A wrong status is worse than none; it can cause real financial harm.

Key principle: “TradeGuard should solve the exception path without damaging the happy path.”

11.4 Analytics Events

Minimum events needed to measure the product. No unnecessary sensitive financial data is collected.

Event	When triggered	Important properties
`trade_review_viewed`	Pre-trade summary is shown	order_type, number_of_legs
`trade_review_confirmed`	User confirms and places the order	order_type, number_of_legs
`order_submitted`	Order is submitted to the engine	order_type, number_of_legs
`order_status_viewed`	User views current order status	status, number_of_legs
`leg_status_updated`	A leg’s state changes	leg_index, leg_status
`unexpected_state_viewed`	User sees an unexpected order state	state_type
`failure_reason_viewed`	User opens the failure explanation	failure_category
`recovery_option_selected`	User selects a recovery action	action_type, order_state
`recovery_completed`	Recovery flow completes	action_type, final_state
`final_trade_state_viewed`	User views the confirmed final state	final_state

11.5 Measurement Funnel

Trade Configured
↓
Trade Review Viewed
↓
Order Submitted
↓

Unexpected State Detected
↓
Unexpected State Viewed
↓
Reason Understood
↓
Recovery Action Selected
↓
Recovery Completed
↓
Final State Confirmed

Drop-off between any two stages pinpoints where users struggle during the exception journey. For example, a gap between **Unexpected State Viewed** and **Reason Understood** points to a communication problem; a gap between **Recovery Action Selected** and **Recovery Completed** points to a broken or confusing recovery flow.

11.6 Experiment Design

A two-stage validation approach, cheap signal first, controlled comparison second.

Stage 1 — Usability Test

Participants: 5 to 8 relevant active / intermediate F&O traders.

Goal: Test whether users can understand the order state, identify the affected leg, understand the failure reason, and identify an appropriate operational next step.

(These tests are proposed, not conducted.)

Stage 2 — Controlled Experiment / A-B Test

- **Control:** Existing order-execution experience.
- **Variant:** TradeGuard clarity experience.
- **Primary success metric:** Unexpected-State Resolution Rate.
- **Secondary metrics:** Time to Understand, Recovery Completion Rate, Unexpected-State Abandonment Rate.
- **Guardrails:** Normal Trade Completion Time, Normal Trade Abandonment Rate, Duplicate Order Attempts, Incorrect Recovery Actions.

11.7 Experiment Hypothesis

Hypothesis (to be validated): “Adding TradeGuard’s execution-status clarity and recovery experience will increase users’ ability to understand and resolve unexpected order states without materially increasing friction during normal successful trades.”

11.8 Decision Rules

Directional rules only, since we have no baseline.

Continue / Scale if: resolution improves, users understand the state faster, recovery completion improves, and guardrails remain healthy.

Iterate if: users understand the problem better, but recovery remains confusing, or the experience adds unnecessary friction.

Kill / Pivot if: there is no meaningful improvement in resolution, users find the experience confusing, or the feature materially harms normal trading performance.

> Numerical success thresholds should be agreed with the product team **before** the experiment begins, using the existing experience as the baseline. This avoids moving the goalposts after seeing the results.

11.9 Key Measurement Principle

> **“Measure the outcome, not the feature.”**

TradeGuard should be judged by whether users can **understand and recover** from unexpected execution states, not by how often they interact with the feature. A quiet TradeGuard that resolves problems fast is more successful than a heavily-used one that keeps users clicking.

12. Interactive Prototype

radeGuard was translated into a four-screen mobile prototype covering the core execution journey:

Pre-Trade Review → Execution Status → Unexpected State → Recovery

The prototype demonstrates how the proposed experience surfaces critical trade information, provides per-leg execution visibility, explains unexpected execution states, and presents operational recovery options while keeping the final trading decision under the user's control.

Interactive Figma Prototype:

https://www.figma.com/design/hQfdWEisGAGQEn7bMBAsgW/Nubra-TradeGuard-—-Execution-Clarity?node-id=0-1&t=mqSX4xeW6IIMvG8p-1